

BLACK SENTINEL

Implementation Plan — Phases 1 & 2

Local-First Personal Privacy Auditor · Windows 11 · Zero Connectivity

PHASE	NAME	MODULES
1	Discovery & Extraction	walker · triage · parsers · decoders
2A	Detection Engine	engine · regex · entropy · presidio · known_pii · validators
2B	Honeycomb System	generator · deployer · watcher · alerter

PHASE 1 — Discovery & Extraction

Filesystem traversal · File classification · Text extraction

WHAT THIS PHASE DOES

- Walk the filesystem → find all files
- Identify each file → decide what to do with it
- Extract readable text → from every file that has it

MODULE STRUCTURE

Module	Description
scanner/walker.py	Traverses filesystem, builds file list
scanner/triage.py	Reads magic bytes, classifies each file
scanner/decoders.py	base64 · hex · url · rot13 · gzip · chardet
scanner/parsers/text_parser.py	.txt .log .csv .json .env etc.
scanner/parsers/archive_parser.py	.zip .rar .7z .tar — extract & re-triage
scanner/parsers/pdf_parser.py	.pdf — text + metadata
scanner/parsers/office_parser.py	.docx .xlsx .pptx
scanner/parsers/sqlite_parser.py	.db .sqlite + browser databases
scanner/parsers/exif_parser.py	.jpg .png .heic .mp4 — metadata only
scanner/parsers/ocr_parser.py	Image files → pytesseract text

walker.py

Recursively walks every accessible folder from the configured root. For each file found, appends its absolute path to a thread-safe queue.

BLACKLISTED FOLDERS — skipped entirely, never entered

Path	Reason
C:\Windows	Core OS files — no PII, all system binaries
C:\Program Files	Installed app binaries — no personal data
C:\Program Files (x86)	32-bit app binaries — same reason
C:\ProgramData\Microsoft	Windows internal data stores
C:\\$Recycle.Bin	Handled separately as a special case
System Volume Information	VSS snapshots — locked by OS
\$* folders	All hidden system folders starting with \$

OUTPUT

```
["C:\\Users\\XYZ\\Documents\\notes.txt",  
 "C:\\Users\\XYZ\\Downloads\\backup.zip",  
 "C:\\Users\\XYZ\\Pictures\\selfie.jpg", ...]
```

triage.py

Reads the first 8 bytes of each file. Matches against known magic byte signatures to determine the true file type — independent of extension — then routes the file to the correct parser or discards it.

MAGIC BYTES TABLE

Magic Bytes	File Type	Action
b'\x4D\x5A'	EXE / DLL	SKIP — system binary
b'\x25\x50\x44\x46'	PDF	→ pdf_parser
b'\x50\x4B\x03\x04'	ZIP / DOCX / XLSX	→ archive_parser or office_parser
b'\x53\x51\x4C\x69'	SQLite	→ sqlite_parser
b'\xFF\xD8\xFF'	JPEG	→ exif_parser + ocr_parser
b'\x89\x50\x4E\x47'	PNG	→ exif_parser + ocr_parser
b'\x52\x61\x72\x21'	RAR	→ archive_parser
b'\x37\x7A\xBC\xAF'	7-Zip	→ archive_parser
b'\x1F\x8B'	GZIP	→ archive_parser
b'-----BEGIN'	Private Key Header	→ text_parser (immediate flag)

FALLBACK — if no magic bytes match

- Read first 512 bytes
- Check for non-text byte values (bytes outside printable ASCII range)
- Text-like → route to text_parser
- Binary-like → SKIP

ROUTING SUMMARY

Type	Files / Source	Action
SKIP	exe · dll · sys · iso · bin · msi	Useless system binaries
TEXT	Readable text	Process with chardet + full detection pipeline
ARCHIVE	zip · rar · 7z · tar · gz	Extract and re-triage all contents
PDF	.pdf	Extract body text + document metadata
OFFICE	docx · xlsx · pptx	Extract text + author metadata
SQLITE	.db .sqlite	Query all tables, all rows, all text columns
IMAGE	jpg · png · heic	EXIF metadata extraction + OCR text extraction
VIDEO	mp4 etc.	Metadata extraction only (pymediainfo)
EMAIL	.eml .msg	Parse body + decode base64 attachments
BROWSER_DB	Chrome / Edge / Firefox	SQLite from known browser profile paths

OUTPUT PER FILE

```
{ "path": "C:\\Users\\arjun\\Downloads\\backup.zip",  
  "type": "archive",  
  "parser": "archive_parser" }
```

parsers/

text_parser.py

- Uses chardet to auto-detect file encoding (UTF-8 / UTF-16 / Latin-1 / Windows-1252)
- Reads file in detected encoding, falls back to UTF-8 with errors='ignore'
- Returns the full decoded string

archive_parser.py

- Extracts to tempfile.mkdtemp() — isolated temp directory
- Each extracted file is passed BACK to triage (recursive handling)
- Depth counter — halts recursion at depth 5 to prevent infinite loops
- Size check — refuses extraction if uncompressed size exceeds 500 MB (zip bomb protection)
- Temp directory deleted immediately after all contents are scanned

pdf_parser.py

- pdfplumber extracts body text page by page
- PyMuPDF (fitz) extracts document metadata: author, creation date, software, revision count
- Both returned as separate text chunks

office_parser.py

- .docx → python-docx: paragraphs + tables + comments + tracked changes + properties
- .xlsx → openpyxl: all cell values across all worksheets + workbook metadata
- .pptx → python-pptx: all slide text + speaker notes + presentation metadata

sqlite_parser.py

- Opens database with built-in sqlite3 module
- SELECT name FROM sqlite_master WHERE type='table' → get all table names
- SELECT * FROM table for each table → iterate all rows
- Concatenate all string/text-type column values
- Return as chunked strings, one chunk per table

exif_parser.py

- Uses exifread library to pull all EXIF tag data
- Priority tags: GPS GPSLatitude/Longitude, Image Make/Model, EXIF LensSerialNumber, Software
- Returns a structured metadata dict — not a text string — flagged directly as findings

ocr_parser.py

- Pillow loads the image file
- Preprocessing: convert to grayscale, apply contrast enhancement
- pytesseract.image_to_string() extracts visible text
- Returns extracted text string — feeds into full detection pipeline

decoders.py

Receives every text chunk from parsers. Runs five decoders in parallel. Each decoder looks for encoded blobs, decodes them, checks if the result is printable text, and if so adds it to the chunk pool. Detection then runs on the original text AND all decoded variants.

Function	Logic
base64_decode()	Find blobs matching <code>[A-Za-z0-9+/]{20,}={0,2}</code> → <code>base64.b64decode()</code> → check <code>.isprintable()</code>
hex_decode()	Find long <code>[0-9a-fA-F]</code> pair strings → <code>bytes.fromhex()</code> → check <code>.isprintable()</code>
url_decode()	Run <code>urlib.parse.unquote()</code> on all text — always applied, not just on matches
rot13_decode()	<code>codecs.decode(text, 'rot_13')</code> — always run, decoded variant always added to pool
gzip_decode()	Detect <code>b'\x1F\x8B'</code> magic inside text blob → <code>gzip.decompress()</code> → decode to string

OUTPUT

```
{ "path": "C:\\Users\\arjun\\config.txt",
  "chunks": [
    "original text content here...",
    "base64 decoded: my_secret_api_key_12345",
    "hex decoded: password123"
  ] }
```

COMPLETE PHASE 1 PIPELINE

- `walker.py` → Produces list of all accessible file paths on the system
- `triage.py` → Classifies each file via magic bytes, routes to correct parser
- `parsers/` → Extracts readable text from every supported file format
- `decoders.py` → Unwraps base64, hex, URL encoding, ROT13, gzip inside text
- **OUTPUT** → list of `(file_path, [text_chunks])` — ready for Phase 2 detection

THREADING MODEL

Files are not processed sequentially. `ThreadPoolExecutor` distributes file processing across all available CPU cores simultaneously:

```
with ThreadPoolExecutor(max_workers=8) as executor:
    futures = {executor.submit(process_file, path): path
               for path in all_files}
```

The walker fills the queue. The thread pool drains it in parallel. On an 8-core machine this produces approximately 6–7x speedup over single-threaded processing.

PHASE 2A — Detection Engine

Regex · Entropy · Presidio NLP · Known PII · Validators

WHAT THIS PHASE DOES

- Receives (file_path, [text_chunks]) from Phase 1
- Runs 4 detectors in parallel on every text chunk
- Validates each finding against format/checksum rules
- Deduplicates, scores severity, produces clean Finding objects

MODULE STRUCTURE

Module	Description
detection/engine.py	Orchestrates all detectors + validation + deduplication
detection/regex_engine.py	30+ named PII patterns with keyword context boosting
detection/entropy_engine.py	Shannon entropy with charset-aware thresholds
detection/presidio_engine.py	Presidio NLP wrapper + custom Indian PII recognizers
detection/known_pii.py	User-provided data matching + hash variants + memory wipe
detection/validators.py	Luhn · Verhoeff · PAN · phone · email format checks

THE FINDING DATA STRUCTURE

Every detector produces Finding objects. This is the single consistent structure that flows through the entire pipeline into the report.

Field	Type / Description
file_path	str — full absolute path to the source file
detector	str — 'regex' / 'entropy' / 'presidio' / 'known_pii'
entity_type	str — 'API_KEY' / 'EMAIL' / 'AADHAAR' / 'CREDIT_CARD' etc.
raw_value	str — the actual matched string (kept in RAM only, never logged)
masked_value	str — 'AKIA****MPLE' — safe version for display in report
context	str — 60 characters either side of the match for human review
confidence	float — 0.0 to 1.0 — from detector or Presidio scorer
severity	str — 'CRITICAL' / 'HIGH' / 'MEDIUM' / 'LOW'
line_number	int — approximate location in source file
validated	bool — True if checksum/format validation passed

engine.py — The Orchestrator

Central coordinator. For each text chunk received from Phase 1, fires all four detectors simultaneously via ThreadPoolExecutor. Collects raw findings, passes each through validation, deduplicates (same value in same file → one finding noting all locations), and assigns final severity.

ORCHESTRATION FLOW

- INPUT → (file_path, ['chunk 1', 'chunk 2', ...]) from Phase 1
- Parallel fire → regex + entropy + presidio + known_pii on each chunk
- validators.py → Each finding individually validated — failures discarded
- Deduplicate → Same value, same file → one entry with all locations
- Score → Final severity assigned based on type + confidence + context
- OUTPUT → Clean list of validated Finding objects

regex_engine.py — Pattern Library

Every PII type gets a named, documented rule. Each rule includes the compiled regex pattern, a confidence score, severity level, and a list of context keywords — if any appear within 100 chars of the match, confidence is boosted by 0.15. This reduces false positives significantly.

RULE STRUCTURE (per pattern)

```
{ "name": "aws_access_key",
  "entity_type": "AWS_ACCESS_KEY",
  "pattern": re.compile(r'AKIA[0-9A-Z]{16}'),
  "confidence": 0.95,
  "severity": "CRITICAL",
  "requires_validation": False,
  "keywords": ["aws", "amazon", "access", "key"] }
```

CREDENTIALS & SECRETS PATTERNS

Rule Name	Pattern
aws_access_key	AKIA[0-9A-Z]{16}
github_pat	ghp_[A-Za-z0-9]{36}
google_api_key	AIzaSy[A-Za-z0-9\-_]{33}
slack_token	xox[baprs]-[A-Za-z0-9\-_]+
stripe_secret_key	sk_live_[A-Za-z0-9]{24}
jwt_token	eyJ[A-Za-z0-9]+\.\eyJ[A-Za-z0-9]+\.[A-Za-z0-9\-_]+
ssh_private_key	-----BEGIN .{0,10}PRIVATE KEY-----
bearer_token	Bearer [A-Za-z0-9\-_]{20,}
db_connection_string	(postgresql mysql mongodb):\//\.\+:\.\+@.+
password_in_config	(password passwd pwd secret)\s*[:=]\s*\S+
generic_api_key	(api_key apikey api_token)\s*[:=]\s*[A-Za-z0-9]{16,}

INDIAN IDENTITY DOCUMENT PATTERNS

Rule Name	Pattern / Logic
aadhaar_number	[2-9]\d{3}\s?\d{4}\s?\d{4} + Verhoeff checksum
pan_card	[A-Z]{5}[0-9]{4}[A-Z]
passport_india	[A-PR-WY][1-9]\d{7}
voter_id	[A-Z]{3}[0-9]{7}

driving_licence	[A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{11}
upi_id	[a-zA-Z0-9.\-_{2,256}@[a-zA-Z]{2,64}
ifsc_code	[A-Z]{4}0[A-Z0-9]{6}

FINANCIAL & CONTACT PATTERNS

Rule Name	Pattern / Logic
credit_card	\b(?:\d[\-]?){13,16}\b + Luhn algorithm validation
bank_account	Keyword near 9-18 digit numeric string
email_address	Standard RFC-compliant email regex
indian_phone	(\+91 0)?[6-9]\d{9}
international_phone	\+?[1-9]\d{6,14}
ip_address_v4	\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b
mac_address	([0-9A-Fa-f]{2}[:\-]){5}[0-9A-Fa-f]{2}
vehicle_registration	[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}

entropy_engine.py — Randomness Detector

Catches secrets that have no recognizable format — machine-generated tokens, unlabelled API keys, cryptographic seeds — by measuring the mathematical randomness (Shannon entropy) of candidate strings.

PROCESSING STEPS

Step	Name	Description
Step 1	Tokenise	Split text on whitespace, quotes, = and : characters. Keep tokens 16–80 chars containing both letters and numbers.
Step 2	Entropy	$H = -\sum p(x) * \log_2(p(x))$ where $p(x)$ = char frequency / length
Step 3	Charset	Detect charset: base64 (threshold 4.5), hex (3.5), alphanumeric (3.8)
Step 4	Context	Nearby keywords (key, secret, token, password) lower threshold by 0.3
Step 5	Confidence	entropy > threshold+1.0 → 0.85 +0.5 → 0.70 +0 → 0.55

ENTROPY REFERENCE VALUES

String Example	Entropy	Interpretation
"hello world"	~2.85	Normal English — not a secret
"password123"	~2.90	Low entropy — weak, may catch with keyword context
"xK9#mP2qL8nR4vB"	~3.80	High entropy — likely a password or key
"AKIADGHE5EXAMPLEKEY"	~4.10	Very high — almost certainly an API key
"eyJhbGciOiJSUzI1Ni..."	~4.60	Maximum — JWT or encoded token

presidio_engine.py — NLP-Based Detection

Thin wrapper around Microsoft Presidio + spaCy en_core_web_lg. Handles what regex cannot: full person names, physical addresses, organisations. Initialised once at startup, cached for the scan session.

INTERNAL PRESIDIO PIPELINE

Stage	Description
NLP Processing	spaCy tokenises text, assigns Part-of-Speech tags to every token
NER Detection	Named Entity Recognition identifies PERSON, ORG, GPE, DATE, MONEY
Regex Recognizers	Presidio's own internal patterns run in parallel with NER
Context Boost	Keywords near matches increase confidence (e.g. 'phone' near a number)
Checksum Validation	Internally validates card numbers, SSN formats, etc.
Confidence Score	Every finding scored 0.0–1.0. Below 0.7 threshold → discarded

CUSTOM RECOGNIZERS (registered at startup)

Recognizer	Logic
AadhaarRecognizer	12-digit pattern + Verhoeff checksum validation
PANRecognizer	ABCDE1234F format + 4th character entity type validation
IFSCRecognizer	Bank code (4) + zero + branch code (6) structure
UPIRecognizer	handle@provider format with Indian VPA rules
VehicleRecognizer	State code + RTO code + series + number format

known_pii.py — Personal Data Matching

Activated only if the user provides their personal data at scan start. Data is held in RAM only — never written to disk. At end of scan, ctypes.memset() overwrites all values in memory before deletion.

WHAT THE USER CAN PROVIDE

Field	Variants Generated
full_name	lowercase, UPPERCASE, first name only, last name only, A. Surname, Name S.
emails	Exact match only — email addresses do not vary significantly
phones	+91 prefix, +91 no space, spaced format, hyphenated, 0-prefix
passwords	plaintext + MD5 + SHA-1 + SHA-256 hash variants via hashlib
aadhaar	Exact match + spaced variants (XXXX XXXX XXXX)
pan	Exact match only — fixed format

MEMORY WIPE ON COMPLETION

```
import ctypes
def wipe(s: str):
    ctypes.memset(id(s), 0, len(s)) # overwrite RAM before del
```

validators.py — False Positive Elimination

Every regex finding with `requires_validation=True` passes through here before being accepted. Findings that fail are silently discarded.

Function	Validates	Logic
<code>luhn_check()</code>	Credit/debit cards	Doubles alternate digits from right, sums all, mod 10 == 0
<code>verhoeff_check()</code>	Aadhaar numbers	Multiplication + permutation table lookup — detects all single-digit errors
<code>pan_check()</code>	PAN cards	4th char encodes entity type (P/C/H/F/A/T/B/L/J/G) — validated
<code>phone_check()</code>	Indian mobile	Must start with 6/7/8/9, exactly 10 digits after country code
<code>email_check()</code>	Email addresses	Domain must have valid TLD, no consecutive dots, valid local part

PHASE 2B — Honeycomb System

Honeytoken generation · Deployment · Live monitoring · Alerting

WHAT THIS PHASE DOES

- Generate realistic fake credentials — structurally identical to real ones
- Deploy them strategically across the filesystem as tripwires
- Monitor those exact files 24/7 via OS filesystem events
- Fire immediate alerts to the UI when any honeytoken is accessed

This system runs completely independently from the detection engine. It starts when the application opens and remains active in the background at all times, even when no scan is running.

MODULE STRUCTURE

Module	Description
honeycomb/generator.py	Creates fake credentials using Python's cryptographic secrets module
honeycomb/deployer.py	Places honeytokens + writes Fernet-encrypted deployment manifest
honeycomb/watcher.py	watchdog FileSystemEventHandler monitoring honeytoken paths only
honeycomb/alerter.py	psutil process ID + PyQt6 signal + encrypted event log

generator.py — Fake Credential Factory

Uses Python's secrets module (cryptographically secure random source) to generate credentials with genuinely high entropy. The result is visually and structurally identical to real credentials — same prefix, same length, same character set — but functionally useless.

GENERATED CREDENTIAL TYPES

Type	Description
Fake AWS Access Key	AKIA + 16 random uppercase alphanumeric chars via secrets.choice()
Fake GitHub PAT	ghp_ + 36 random alphanumeric chars
Fake Generic API Key	32-char high-entropy string, mixed case + digits
Fake .env file	Multi-line file: AWS key + secret + DB URL + Stripe key + GitHub token
Fake SSH Private Key	Proper BEGIN/END headers with realistic base64-encoded body
Fake credentials.json	JSON: api_key + client_secret + database_password fields
Fake config.ini	[credentials] section with username + password entries

EXAMPLE GENERATED .env HONEYTOKEN

```
AWS_ACCESS_KEY_ID=AKIADGHE5JEXAMPLEKEY00
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYFAKEKEY
DATABASE_URL=postgresql://admin:fakepassword123@localhost:5432/proddb
STRIPE_SECRET_KEY=sk_live_FakeKeyHereForTripwire0000
GITHUB_TOKEN=ghp_FakeGithubTokenForMonitoring123456
```

deployer.py — Strategic Placement

Places honeypot files at locations an attacker or malicious credential scanner would check first.
Records all deployments in an encrypted manifest.

DEPLOYMENT LOCATIONS

Path	Rationale
Documents\credentials.txt	Highest-value target location
Desktop\.env	Developer credential file, high attacker priority
.ssh\backup_key	SSH key store — attackers always check .ssh/
AppData\Roaming\app_config.json	App data config — commonly scraped
Downloads\api_keys.txt	Downloads folder — often overlooked by owners
Documents\dev\secrets.env	Developer subfolder — targeted by scanners

THE MANIFEST — stored Fernet-encrypted

```
{ "deployed_tokens": [  
  { "path": "C:\\Users\\arjun\\Desktop\\.env",  
    "token_id": "uuid-v4-here",  
    "deployed_at": "2025-06-01T14:32:11",  
    "content_hash": "sha256-of-file-content" }  
]
```

Fernet encryption: from cryptography.fernet import Fernet. Key generated once and stored in app data.
Manifest encrypted with fernet.encrypt() — cannot be read without the key.

watcher.py — The Tripwire

Uses watchdog's FileSystemEventHandler with Windows native ReadDirectoryChangesW under the hood. Watches ONLY the specific honeypot file paths — not the entire filesystem — so CPU cost while idle is effectively zero.

MONITORED EVENTS

Event	Description
on_modified()	File content was changed — possible tampering or exfiltration attempt
on_opened()	File was opened/read — primary tripwire event (Windows supports this)
on_deleted()	Honeypot was deleted — attacker may have removed evidence
on_moved()	File was moved or renamed — evasion attempt

THREADING MODEL

- Watcher runs on its own dedicated daemon thread
- Starts at application launch — independent of scan state
- Uses Python threading.Event for clean shutdown on app close
- Completely separate from scanner ThreadPoolExecutor

alerter.py — Notification & Logging

When the watcher fires, the alerter executes four actions in sequence:

ALERT SEQUENCE

Step	Action	Detail
1	Record event	Timestamp + honeypath path + event type → alert_record dict
2	Identify process	psutil.process_iter(['open_files']) → match path → get name + PID
3	Signal PyQt6 UI	Qt signal emitted from watcher thread → safely received on main UI thread
4	Log to disk	Alert record appended to Fernet-encrypted local log file

UI ALERT DISPLAY

HONEYTOKEN ACCESSED File: C:\Users\arjun\Desktop\.env By: python.exe (PID: 4821) Time: 14:32:11 Alert: A process on your system is scanning for credentials

COMPLETE PHASE 2 PIPELINE

- Phase 1 output → (file_path, [text_chunks]) — input to detection engine
- engine.py → Receives chunks, fires all 4 detectors in parallel per chunk
- regex_engine → Pattern matching: 30+ named rules across all PII categories
- entropy_engine → Shannon entropy analysis: catches unlabelled secrets
- presidio_engine → NLP: names, addresses, organisations + custom Indian PII
- known_pii → Personal data matching + hash comparison + RAM wipe on done
- validators.py → Luhn + Verhoeff + format checks: discard false positives
- OUTPUT → Validated Finding list → deduplicated → scored → Phase 3 report

HONEYCOMB SYSTEM — ALWAYS ACTIVE

- generator.py → Creates fake credentials indistinguishable from real ones
- deployer.py → Places honeypaths at 6+ strategic paths + encrypts manifest
- watcher.py → Daemon thread: monitors honeypath paths via OS filesystem events
- alerter.py → psutil process lookup → Qt signal to UI → encrypted log append